Testen mit JUnit (JUnit-Basics)

Carsten Gips (HSBI)

TL;DR
In JUnit 4 und 5 werden Testmethoden mit Hilfe der Annotation `@Test` ausgezeichnet. Über die verschiedenen `assert*()`-Methoden kann das Testergebnis mit dem erwarteten Ergebnis verglichen werden und entsprechend ist der Test "grün" oder "rot". Mit den verschiedenen `assume*()`-Methoden kann dagegen geprüft werden, ob eventuelle Vorbedingungen für das Ausführen eines Testfalls erfüllt sind - anderenfalls wird der Testfall dann übersprungen.
Mit Hilfe von `@Before` und `@After` können Methoden gekennzeichnet werden, die jeweils vor jeder Testmethode und nach jeder Testmethode aufgerufen werden. Damit kann man seine Testumgebung auf- und auch wieder abbauen (JUnit 4).
Erwartete Exceptions lassen sich in JUnit 4 mit einem Parameter `expected` in der Annotation `@Test` automatisch prüfen: `@Test(expected=package.Exception.class)`. In JUnit 4 besteht die Möglichkeit, Testklassen zu Testsuiten zusammenzufassen und gemeinsam laufen zu lassen.
Videos

- VL JUnit Basics
- Demo assume() vs. assert()
- Demo Parametrisierte Tests mit JUnit4
- Demo Parametrisierte Tests mit JUnit5

---

1 JUnit: Ergebnis prüfen

Klasse `org.junit.Assert` enthält diverse **statische** Methoden zum Prüfen:

```
// Argument muss true bzw. false sein
void assertTrue(boolean);
void assertFalse(boolean);

// Gleichheit im Sinne von equals()
void assertEquals(Object, Object);

// Test sofort fehlschlagen lassen
void fail();

...
```

2 To "assert" or to "assume"?

- Mit `assert*` werden Testergebnisse geprüft
  - Test wird ausgeführt
  - Ergebnis: OK, Failure, Error

- Mit `assume*` werden Annahmen über den Zustand geprüft
  - Test wird abgebrochen, wenn Annahme nicht erfüllt
  - Prüfen von Vorbedingungen: Ist der Test hier ausführbar/anwendbar?

Beispiel: junit4.TestAssume

3 Setup und Teardown: Testübergreifende Konfiguration

```
private Studi x;

@Before
public void setUp() { x = new Studi(); }

@Test
public void testToString() {
    // Studi x = new Studi();
    assertEquals(x.toString(), "Heinz (15cps)");
}
```

`@Before` wird **vor jeder** Testmethode aufgerufen
`@BeforeClass` wird **einmalig** vor allen Tests aufgerufen (**`static`**!)
`@After` wird **nach jeder** Testmethode aufgerufen
`@AfterClass` wird **einmalig** nach allen Tests aufgerufen (**`static`**!)

In JUnit 5 wurden die Namen dieser Annotationen leicht geändert:

JUnit 4	JUnit 5
`@Before`	`@BeforeEach`
`@After`	`@AfterEach`
`@BeforeClass`	`@BeforeAll`
`@AfterClass`	`@AfterAll`

4 Beispiel für den Einsatz von `@Before`

Annahme: **alle/viele** Testmethoden brauchen **neues** Objekt `x` vom Typ `Studi`

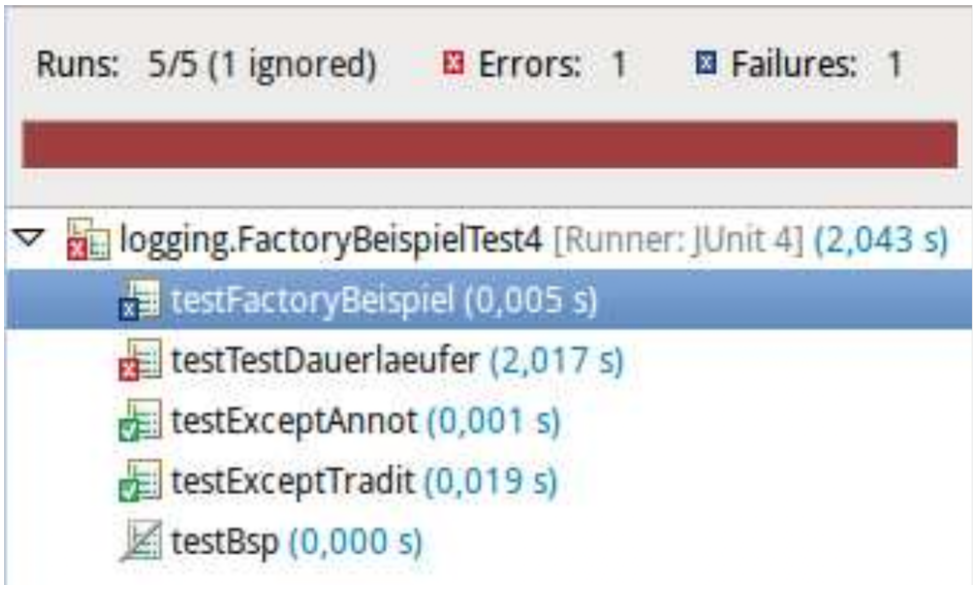
```
private Studi x;

@Before
public void setUp() {
    x = new Studi("Heinz", 15);
}

```

```
8  @Test
9  public void testToString() {
10     // Studi x = new Studi("Heinz", 15);
11     assertEquals(x.toString(), "Name: Heinz, credits: 15");
12 }
13
14 @Test
15 public void testGetName() {
16     // Studi x = new Studi("Heinz", 15);
17     assertEquals(x.getName(), "Heinz");
18 }
```

5 Ignorieren von Tests

- Hinzufügen der Annotation `@Ignore`
- Alternativ mit Kommentar: `@Ignore("Erst im nächsten Release")`

```
1 @Ignore("Warum ignoriert")
2 @Test
3 public void testBsp() {
4     Bsp x = new Bsp();
5     assertTrue(x.isTrue());
6 }
```

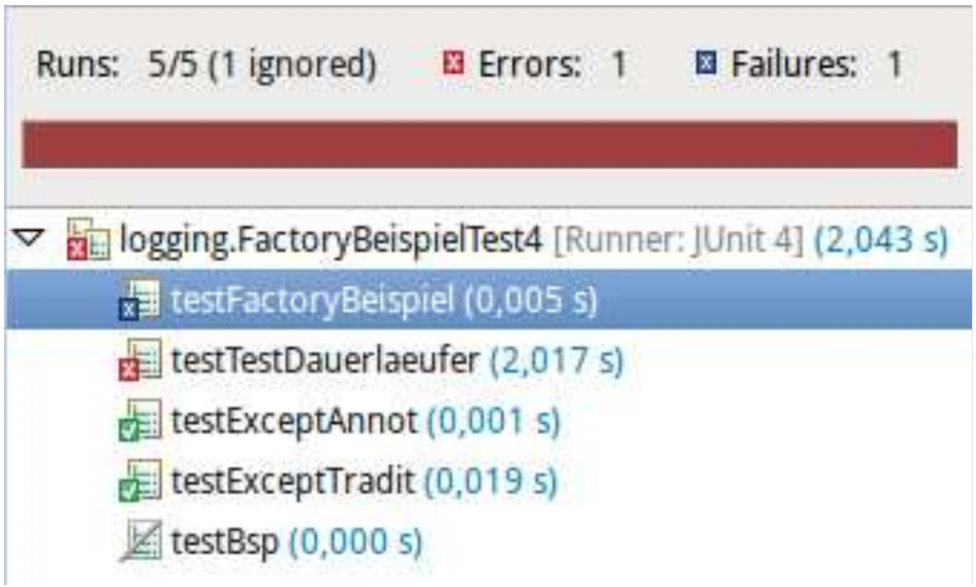


In JUnit 5 wird statt der Annotation `@Ignore` die Annotation `@Disabled` mit der selben Bedeutung verwendet. Auch hier lässt sich als Parameter ein String mit dem Grund für das Ignorieren des Tests hinterlegen.

6 Vermeidung von Endlosschleifen: Timeout

- Testfälle werden nacheinander ausgeführt
- Test mit Endlosschleife würde restliche Tests blockieren
- Erweitern der `@Test`-Annotation mit Parameter "`timeout`":
  => `@Test(timeout=2000)` (Zeitangabe in Millisekunden)

```
@Test(timeout = 2000)
void testTestDauerlaeufer() {
    while (true) { ; }
}
```



In JUnit 5 hat die Annotation `@Test` keinen `timeout`-Parameter mehr. Als Alternative bietet sich der Einsatz von `org.junit.jupiter.api.Assertions.assertTimeout` an. Dabei benötigt man allerdings *Lambda-Ausdrücke* (Verweis auf spätere VL):

```
@Test
void testTestDauerlaeufer() {
    assertTimeout(ofMillis(2000), () -> {
        while (true) { ; }
    });
}
```

(Beispiel von oben mit Hilfe von JUnit 5 formuliert)

7 Test von Exceptions: Expected

Traditionelles Testen von Exceptions mit **`try`** und **`catch`**:

```
@Test
public void testExceptTradit() {
    try {
        int i = 0 / 0;
        fail("keine ArithmeticException ausgeloest");
    } catch (ArithmeticException aex) {
        assertNotNull(aex.getMessage());
    } catch (Exception e) {
        fail("falsche Exception geworfen");
    }
}
```

Der `expected`-Parameter für die `@Test`-Annotation in JUnit 4 macht dies deutlich einfacher: `@Test(expected = MyException.class)` => Test scheitert, wenn diese Exception **nicht** geworfen wird

```
@Test(expected = java.lang.ArithmeticException.class)
public void testExceptAnnot() {
    int i = 0 / 0;
}
```

In JUnit 5 hat die Annotation `@Test` keinen `expected`-Parameter mehr. Als Alternative bietet sich der Einsatz von `org.junit.jupiter.api.Assertions.assertThrows` an. Dabei benötigt man allerdings *Lambda-Ausdrücke* (Verweis auf spätere VL):

```
@Test
public void testExceptAnnot() {
    assertThrows(java.lang.ArithmeticException.class, () -> {
        int i = 0 / 0;
    });
}
```

(Beispiel von oben mit Hilfe von JUnit 5 formuliert)

8 Parametrisierte Tests

Manchmal möchte man den selben Testfall mehrfach mit anderen Werten (Parametern) durchführen.

```
class Sum {
    public int sum(int i, int j) {
        return i + j;
    }
}

class SumTest {
    @Test
    public void testSum() {
        Sum s = new Sum();
        assertEquals(s.sum(1, 1), 2);
    }
    // und mit (2,2, 4), (2,2, 5), ...????
}
```

Prinzipiell könnte man dafür entweder in einem Testfall eine Schleife schreiben, die über die verschiedenen Parameter iteriert. In der Schleife würde dann jeweils der Aufruf der zu testenden Methode und das gewünschte Assert passieren. Alternativ könnte man den Testfall entsprechend oft duplizieren mit jeweils den gewünschten Werten.

Beide Vorgehensweisen haben Probleme: Im ersten Fall würde die Schleife bei einem Fehler oder unerwarteten Ergebnis abbrechen, ohne dass die restlichen Tests (Werte) noch durchgeführt würden. Im zweiten Fall bekommt man eine unnötig große Anzahl an Testmethoden, die bis auf die jeweiligen Werte identisch sind (Code-Duplizierung).

8.1 Parametrisierte Tests mit JUnit 4

JUnit 4 bietet für dieses Problem sogenannte "parametrisierte Tests" an. Dafür muss eine Testklasse in JUnit 4 folgende Bedingungen erfüllen:

1. Die Testklasse wird mit der Annotation `@RunWith(Parameterized.class)` ausgezeichnet.
2. Es muss eine öffentliche statische Methode geben mit der Annotation `@Parameters`. Diese Methode liefert eine Collection zurück, wobei jedes Element dieser Collection ein Array mit den Parametern für einen Durchlauf der Testmethoden ist.
3. Die Parameter müssen gesetzt werden. Dafür gibt es zwei Varianten:
   a) Für jeden Parameter gibt es ein öffentliches Attribut. Diese Attribute müssen mit der Annotation `@Parameter` markiert sein und können in den Testmethoden normal genutzt werden. JUnit sorgt dafür, dass für jeden Eintrag in der Collection aus der statischen `@Parameters`-Methode diese Felder gesetzt werden und die Testmethoden aufgerufen werden.
   b) Alternativ gibt es einen Konstruktor, der diese Werte setzt. Die Anzahl der Parameter im Konstruktor muss dabei exakt der Anzahl (und Reihenfolge) der Werte in jedem Array in der von der statischen `@Parameters`-Methode gelieferten Collection entsprechen. Der Konstruktor wird für jeden Parametersatz einmal aufgerufen und die Testmethoden einmal durchgeführt.

Letztlich wird damit das Kreuzprodukt aus Testmethoden und Testdaten durchgeführt.

Beispiel: junit4.SumTestConstructor, junit4.SumTestParameters

8.2 Parametrisierte Tests mit JUnit 5

In JUnit 5 werden parametrisierte Tests mit der Annotation `@ParameterizedTest` gekennzeichnet (statt mit `@Test`).

Mit Hilfe von `@ValueSource` kann man ein einfaches Array von Werten (Strings oder primitive Datentypen) angeben, mit denen der Test ausgeführt wird. Dazu bekommt die Testmethode einen entsprechenden passenden Parameter:

```
1 @ParameterizedTest
2 @ValueSource(strings = {"wuppie", "fluppie", "foo"})
3 void testWuppie(String candidate) {
4     assertTrue(candidate.equals("wuppie"));
5 }
```

Alternativ lassen sich als Parameterquelle u.a. Aufzählungen (`@EnumSource`) oder Methoden (`@MethodSource`) angeben.

Hinweis: Parametrisierte Tests werden in JUnit 5 derzeit noch als "*experimentell*" angesehen!

Beispiel: junit5.TestValueSource, junit5.TestMethodSource

9 Testsuiten: Tests gemeinsam ausführen (JUnit 4)

Eclipse: `New` > `Other` > `Java` > `JUnit` > `JUnit Test Suite`

```
import org.junit.runner.RunWith;
import org.junit.runners.Suite;
import org.junit.runners.Suite.SuiteClasses;

@RunWith(Suite.class)
@SuiteClasses({
    // Hier kommen alle Testklassen rein
    PersonTest.class,
    StudiTest.class
})

public class MyTestSuite {
    // bleibt leer!!!
}
```

10 Testsuiten mit JUnit 5

In JUnit 5 gibt es zwei Möglichkeiten, Testsuiten zu erstellen:

- `@SelectPackages`: Angabe der Packages, die für die Testsuite zusammengefasst werden sollen
- `@SelectClasses`: Angabe der Klassen, die für die Testsuite zusammengefasst werden sollen

```
@RunWith(JUnitPlatform.class)
@SelectClasses({StudiTest5.class, WuppieTest5.class})
public class MyTestSuite5 {
    // bleibt leer!!!
}
```

Zusätzlich kann man beispielsweise mit `@IncludeTags` oder `@ExcludeTags` Testmethoden mit bestimmten Tags einbinden oder ausschließen. Beispiel: Schließe alle Tests mit Tag "develop" aus: `@ExcludeTags("develop")`. Dabei wird an den Testmethoden zusätzlich das Tag `@Tag` verwendet, etwas `@Tag("develop")`.

Achtung: Laut der offiziellen Dokumentation (Abschnitt "4.4.4. Test Suite") gilt zumindest bei der Selection über `@SelectPackages` der Zwang zu einer Namenskonvention: Es werden dabei nur Klassen gefunden, deren Name mit `Test` beginnt oder endet! Weiterhin werden Testsuites mit der Annotation `@RunWith(JUnitPlatform.class)` **nicht** auf der "JUnit 5"-Plattform ausgeführt, sondern mit der JUnit 4-Infrastuktur!

11 Wrap-Up

JUnit als Framework für (Unit-) Tests; hier JUnit 4 (mit Ausblick auf JUnit 5)

- Testmethoden mit Annotation `@Test`
- `assert` (Testergebnis) vs. `assume` (Testvorbedingung)
- Aufbau der Testumgebung `@Before`
- Abbau der Testumgebung `@After`
- Steuern von Tests mit `@Ignore` oder `@Test(timout=XXX)`
- Exceptions einfordern mit `@Test(expected=package.Exception.class)`
- Tests zusammenfassen zu Testsuiten

12 Zum Nachlesen

- vogella GmbH (2021)
- The JUnit Team (2022)
- Kleuker (2019)
- Osherove (2014)
- Spillner und Linz (2012)
- Thies, Noelke, und Ungerc (o. J.)

Lernziele

- k3: Steuern von Tests (ignorieren, zeitliche Begrenzung)
- k3: Prüfung von Exceptions
- k3: Aufbau von Testsuiten mit JUnit

Quizzes

- Quiz JUnit-Basics (ILIAS)

Challenges

Schreiben Sie eine JUnit-Testklasse (JUnit 4.x oder 5.x) und testen Sie eine `ArrayList<String>`. Prüfen Sie dabei, ob das Einfügen und Entfernen wie erwartet funktioniert.

1. Initialisieren Sie in einer `setUp()`-Methode das Testobjekt und fügen Sie zwei Elemente ein. Stellen Sie mit einer passenden `assume*`-Methode sicher, dass die Liste genau diese beiden Elemente enthält. Die `setUp()`-Methode soll vor jedem Testfall ausgeführt werden.
2. Setzen Sie in einer `tearDown()`-Methode das Testobjekt wieder auf **`null`** und stellen Sie mit einer passenden `assume*`-Methode sicher, dass das Testobjekt tatsächlich **`null`** ist. Die `tearDown()`-Methode soll nach jedem Testfall ausgeführt werden.
3. Schreiben Sie eine Testmethode `testAdd()`. Fügen Sie ein weiteres Element zum Testobjekt hinzu und prüfen Sie mit einer passenden `assert*`-Methode, ob die Liste nach dem Einfügen den gewünschten Zustand hat: Die Länge der Liste muss 3 Elemente betragen und alle Elemente müssen in der richtigen Reihenfolge in der Liste stehen.
4. Schreiben Sie eine Testmethode `testRemoveObject()`. Entfernen Sie ein vorhandenes Element (über die Referenz auf das Objekt) aus dem Testobjekt und prüfen Sie mit einer passenden `assert*`-Methode, ob die Liste nach dem Entfernen den gewünschten Zustand hat: Die Liste darf nur noch das verbleibende Element enthalten.
5. Schreiben Sie eine Testmethode `testRemoveIndex()`. Entfernen Sie ein vorhandenes Element über dessen *Index* in der Liste und prüfen Sie mit einer passenden `assert*`-Methode, ob die Liste nach dem Entfernen den gewünschten Zustand hat: Die Liste darf nur noch das verbleibende Element enthalten. (Nutzen Sie zum Entfernen die `remove(int)`-Methode der Liste.)
6. Schreiben Sie zusätzlich einen **parametrisierten JUnit-Test** für die folgende Klasse:

```java
import java.util.ArrayList;

public class SpecialArrayList extends ArrayList<String> {
    public void concatAddStrings(String a, String b) {
        this.add(a + b);
    }
}
```

Testen Sie, ob die Methode `concatAddStrings` der Klasse `SpecialArrayList` die beiden übergebenen Strings korrekt konkateniert und das Ergebnis richtig in die Liste einfügt. Testen Sie dabei mit mindestens den folgenden Parameter-Tripeln:

a	b	expected
“”	“”	“”
“”	“a”	“a”
“a”	“”	“a”
“abc”	“123”	“abc123”

Quellen

Kleuker, S. 2019. *Qualitätssicherung durch Softwaretests*. Springer Vieweg. https://doi.org/10.1007/978-3-658-24886-4.

Osherove, R. 2014. *The Art of Unit Testing*. Manning.

Spillner, A., und T. Linz. 2012. *Basiswissen Softwaretest*. 5. Aufl. dpunkt.

The JUnit Team. 2022. „JUnit 5“. 2022. https://junit.org/.

Thies, A., C. Noelke, und Ungerc. o. J. „Einführung in JUnit“. Fernuniversität in Hagen. Zugegriffen 14. April 2020. https://wiki.fernuni-hagen.de/eclipse/index.php/EinfÃijhrung_in_JUnit.

vogella GmbH. 2021. „JUnit 5 Tutorial - Learn How to Write Unit Tests“. 2021. https://www.vogella.com/tutorials/JUnit/article.html.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 71232c0 (tooling: shift headings (use h1 as top-level headings), 2025-04-29)